

agentr: Practical Scaffolding for Coding-AI Workflows

Oliver Zhou

Lily Li

Abstract

`agentr` is an R package for specifying, reviewing, and scaffolding agentic AI workflows. Its practical contribution is not to provide another model wrapper, chat interface, or autonomous execution engine. Instead, it supplies a structured layer for using external coding AIs—including repository-aware coding assistants, command-line coding agents, and GUI/API-backed model tools—inside inspectable development workflows. `agentr` standardizes task-local artifacts for workflow design, memory, curated knowledge, interface expectations, autonomy boundaries, and visual review. These artifacts make it easier for a coding assistant to inspect a project, infer or revise a design, render review pages, and implement code against approved specifications while preserving human review, provenance, and version history. The package is especially suited to R-centered, repository-based work in which model assistance is valuable but intermediate decisions must remain visible, editable, and auditable.

Introduction

Many AI-assisted workflows still resemble a direct path from a user’s natural-language request to a final artifact: a paper draft, an analytical chart, a processed data set, a script, or a software patch. This path is convenient, but it hides much of the work that matters. The decomposition of the task, the assumptions used by the model, the handoff between deterministic code and stochastic LLM judgment, and the authority given to the model are often buried in a chat transcript or lost altogether.

This problem is becoming more important as LLM-based agents move from prompt-response assistance toward systems that plan, use tools, retain memory, coordinate with other agents, and act across longer workflows (Xi et al. 2023; Ferrag, Tihanyi, and Debbah 2025; Derouiche, Brahmi, and Mazeni 2025). Across domains such as education, cybersecurity, business process management, anomaly management, credit risk, and software quality assurance, recent agentic AI proposals share a similar promise: task decomposition, tool use, feedback, and specialized agents can make complex work more adaptive and scalable (Kamalov et al. 2025; Lazer et al. 2026; Dumas, Milani, and Chapela-Campa 2026; Barenji and Khoshgoftar 2025; Kubam 2025; Naqvi, Baqar, and Mohammad 2026). They also share a similar risk profile. Once systems combine planning, memory, external tools, and multi-step execution, failures are no longer isolated model outputs. They can propagate through workflows, become difficult to reconstruct, and create accountability gaps (Zhu et al. 2026; Ghosh et al. 2025).

`agentr` responds to this practical tension from the standpoint of day-to-day coding-AI development. It treats

intermediate design as a first-class object. A task is represented by editable specifications: workflow nodes and edges, gates, schemas, memory fields, curated knowledge, interface expectations, autonomy stage, and review artifacts. External coding AIs can still do substantial work, but their work is organized around files that can be inspected, validated, revised, and committed to Git. The goal is not to reduce AI use. The goal is to make AI-assisted development more structural, transparent, and portable.

In one sentence: `agentr` is a practical scaffolding framework for making coding-AI-driven work structured, inspectable, and portable.

The core problem: capable coding AIs without reliable intermediate structure

Modern coding assistants are increasingly capable of inspecting repositories, writing code, editing files, running tests, and revising implementations. This capability creates a practical tension. On one side, developers can delegate more implementation work than before. On the other side, a direct natural-language-to-output workflow gives users limited control over intermediate decisions. It is often unclear which parts of a task were deterministic, which parts depended on model judgment, which assumptions were treated as approved, and where a human should have intervened.

This is especially problematic for analytical, scientific, financial, policy, and technical-writing work. In those settings, the final output matters only if the path to it can be inspected. A chart is not enough; the user needs to know what data were used and what transformations were applied. A report is not enough; the user needs to know which assumptions, heuristics, and exceptions shaped the argument. A software patch is not enough; the user needs to know which design was approved and whether the implementation matched it.

The difficulty is not simply that LLMs can make mistakes. It is that agentic workflows distribute mistakes across plans, memory, tool calls, intermediate files, and human approvals. Surveys of LLM agents identify planning, reflection, tool use, memory, action, and multi-agent coordination as core capabilities, but these same capabilities introduce evaluation, security, interoperability, and governance challenges (Huang et al. 2024; Du 2026; Derouiche, Brahmi, and Mazeni 2025). Work on system-level interpretability argues that agentic systems require trajectory-level provenance and causal tracing rather than only model-level explanations (Zhu et al. 2026). Work on human-agent systems similarly argues that selective human involvement should be designed into workflows rather than treated as an afterthought (Zou et al. 2025; Feng, McDonald, and Zhang 2025).

`agentr` addresses this problem by creating a reviewable layer between human intent and downstream execution. Instead of asking the coding AI to go directly from instruction to final result, `agentr` asks it to work through explicit artifacts. These artifacts provide more structural control than ordinary prompting, better provenance than a chat-only workflow, and more flexibility in assigning decision-making authority among humans, deterministic code, external LLMs, and downstream runtimes.

The main use case: coding-assistant scaffolding

The strongest current use case for `agentr` is developer-facing scaffolding with external coding AIs. A typical workflow is:

```
existing task code and docs
-> coding assistant reads agentr guidance
-> assistant infers task-local YAML specs
-> agentr validates and renders review artifacts
-> human reviews specs, diagrams, assumptions, and outputs
-> assistant revises specs or implementation
-> Git records the evolution
```

This pattern treats coding AIs as powerful collaborators rather than invisible decision-makers. The human does not need to hand-write every line of implementation code. At the same time, the human is not reduced to passively accepting a final output. The task design becomes visible through workflow graphs, memory schemas, knowledge items, review notes, and file diffs.¹

The point is not that `agentr` guarantees that a coding AI will always follow instructions. In practice, coding assistants may ignore guidance, over-implement ambiguous requirements, skip validation steps, or treat provisional assumptions as approved. `agentr` reduces these risks by moving guidance out of a transient chat message and into project-local, machine-readable, human-reviewable artifacts. The coding AI can be instructed to modify a specific specification, render the corresponding review page, and implement only after review. Validation checks and Git diffs then provide an additional layer of discipline: if the implementation diverges from the approved design, the divergence becomes easier to see.

This emphasis is aligned with recent work showing that agent performance depends strongly on the surrounding interface and harness, not only on the model. ReAct showed the value of interleaving reasoning and action; SWE-agent showed that software-engineering agents benefit from interfaces designed around repository search, editing feedback, guardrails, and history management; and recent work on code as an agent harness frames executable, inspectable code artifacts as a substrate for reliable agent behavior (Yao et al. 2022; Yang et al. 2024; Ning et al. 2026). `agentr` extends this idea at the project-scaffolding level: the important interface is not only the shell, editor, or tool API, but also the structured specification layer that tells coding AIs what work is being done, which parts are approved, and what evidence must remain after the work is changed.

Standardized artifacts as cognitive and review scaffolding

`agentr` organizes agentic designs around a small set of standardized artifacts. The purpose of these artifacts is not to force every project into one runtime architecture. Their purpose is to make the cognitive structure of a task explicit enough for humans and coding AIs to inspect, revise, and reuse.

¹A secondary proposal-state mode can preserve constrained suggestions, discussion states, and approval status inside R when Git is not the primary review system. In this paper, however, proposal state is treated as supporting functionality rather than the central contribution.

The workflow artifact captures procedural knowledge: what the task does, how steps depend on one another, where review gates exist, and which details still require human judgment or external implementation. The memory artifact describes what the agent or workflow should remember, including context, semantic, episodic, and procedural memory. The knowledge artifact captures curated domain content such as concepts, causal beliefs, rules, exceptions, heuristics, evaluation criteria, style preferences, and risk warnings. The review artifact turns these pieces into a portable page that can show workflow graphs, memory structure, narrative knowledge, graph-shaped knowledge, implementation notes, warnings, and feedback fields. At the highest level, an agent specification can bundle the task, workflow, memory, knowledge, state requirements, interfaces, autonomy stage, implementation targets, and metadata.

This is the paper’s main claim about standardization: `agentr` standardizes enough structure for coding-AI collaboration without trying to become the final execution runtime. The package therefore sits between ordinary prompting and full agent-platform design. Compared with chat-only workflows, it makes assumptions, dependencies, gates, and knowledge references explicit. Compared with provider-specific agent frameworks, it keeps the design portable because the key artifacts are files that can be reviewed, rendered, and versioned independently of any one model provider.

This design connects to several strands of the agentic AI literature. Design-pattern approaches argue that agentic systems benefit from reusable abstractions for roles, policies, human oversight, orchestration, and accountability (Milosevic and Rabhi 2026; Wray, Kirk, and Laird 2025). Work on open agent specifications and reusable workflow frameworks similarly argues for declarative or standardized representations that separate workflow design from framework-specific implementation (Amini et al. 2025; Gaoyang Zhang et al. 2026). Work on knowledge protocols and expert-knowledge codification emphasizes that expert behavior often depends on explicit procedural rules and tacit domain knowledge that must be structured before it can reliably guide LLM behavior (Guangwei Zhang 2025; Uulu et al. 2026). `agentr` adopts a practical, project-local version of these ideas for R-centered coding workflows.

The cognitive core of `agentr` is therefore not a separate theoretical module. It is the same standardized specification layer viewed as cognitive infrastructure. Workflow structure externalizes planning. Memory schema externalizes what should persist across interactions. Knowledge schema externalizes domain assumptions and evaluation criteria. Interface specifications externalize what tools and files the coding AI or downstream runtime may touch. Autonomy labels externalize who has authority at each step. Review artifacts externalize the evidence needed for human judgment. Together, these objects transform a vague instruction into a reviewable cognitive scaffold.

Visual review as a governance interface

A central design choice in `agentr` is to render specifications into visual review artifacts. Visual review matters because structured files are useful only if humans can inspect them without reading every YAML field. Workflow graphs can show decomposition, ordering, gates, and dependencies. Memory and knowledge panels can show what context the agent is expected to retain and which assumptions support each workflow step. Review notes can expose weakly specified nodes, unresolved questions, missing schemas, unapproved knowledge, or places where automation is inappropriate.

This review layer responds to a recurring challenge in human-AI collaboration: humans need enough visibility to exercise judgment, but too much raw detail can make review impractical. Prior work comparing human and AI workflows suggests that agents often prefer programmatic procedures while humans rely more heavily on visual grounding, practical judgment, instruction revisiting, and verification (Wang et al. 2025). `agentr` uses visual review as a boundary object between these modes. The coding AI can operate on structured files; the human can inspect rendered graphs and summaries; both can refer back to the same project artifacts.

Visual review also has limitations. A diagram can create false confidence if it is stale, oversimplified, or disconnected from the implementation. A polished review page can hide missing validations or unclear authority boundaries. `agentr` therefore treats visual review as an inspectability aid, not as proof of correctness. The remedy is to keep review pages generated from source specifications, expose warnings and validation results, preserve file paths and implementation notes, and use Git history to compare specification changes with code changes. In this sense, visual review complements rather than replaces tests, code review, and human domain judgment.

Why this design fits current coding-AI development

Recent agent research increasingly treats LLM systems as more than prompt-response generators. Agents are commonly described as systems that combine perception, reasoning, planning, memory, tool use, action, reflection, and feedback (Xi et al. 2023; Ferrag, Tihanyi, and Debbah 2025). In software engineering, the key question is not only whether a model can generate code, but whether the surrounding environment helps the model search, edit, test, recover, and preserve the right context (Yang et al. 2024). More generally, code-centered agent harnesses suggest that durable artifacts, executable feedback, state, and verification can make agentic work more reliable than ungrounded text generation alone (Ning et al. 2026).

`agentr` fits this direction, but from a practical R-package and project-scaffolding perspective. It does not attempt to define the final runtime for every agent. Instead, it supplies the artifacts that make coding-AI work easier to inspect and govern. The coding assistant can be external and replaceable. The runtime can live in another package or system. The important contribution is the specification and review boundary: what the task is supposed to do, what the model is allowed to decide, what humans must approve, and what evidence remains after revision.

This positioning is especially useful while coding AIs are evolving quickly. If a framework binds itself too tightly to one provider, model, or runtime loop, it may become obsolete as tools change. `agentr` instead aims to standardize the project-local shapes that survive tool churn: YAML specifications, rendered graphs, review HTML, validated design objects, implementation hints, and Git-tracked evolution. This is consistent with the broader movement toward framework-agnostic specifications, interoperable agent protocols, and production-grade workflow discipline (Derouiche, Brahmi, and Mazeni 2025; Amini et al. 2025; Bandara et al. 2025).

Suitable users, tasks, and fields

`agentr` is most useful for developers, analysts, researchers, and small technical teams who already work with code repositories, R projects, scripts, data folders, Quarto documents, or computational notebooks. These users want the productivity of coding AIs, but they also need an inspectable path from goal to output.

The best tasks are multi-step, repeatable, and decomposable. They usually include some combination of data ingestion, transformation, interpretation, charting, report writing, validation, and human review. They benefit from explicit workflow nodes, visible gates, reusable memory, curated knowledge, documented assumptions, and rendered review pages. They are also tasks where implementation can be assisted by coding AIs but should not be treated as a black box.

Good application scenarios include:

- data-analysis and reporting pipelines;
- chart and dashboard generation;
- research-assistant workflows;
- computational social science and economic analysis;
- financial or policy analysis where assumptions and exceptions matter;
- package maintenance and repository-level code revision;
- technical publishing with Quarto or R Markdown;
- internal business automation where review and provenance are required.

`agentr` is less useful for one-off casual chats, simple prompt templates, real-time conversational products, or applications that mainly need a provider-specific API client. It is also not meant to be the communication backend, the messaging layer, or the autonomous execution engine. Those concerns can live in adjacent packages or downstream systems.

Package boundary and contribution

The package boundary is intentionally modest. `agentr` supplies specification, scaffolding, validation, rendering, proposal support, and review. It does not claim to solve every problem in autonomous agency. It does not replace model providers, agent runtimes, execution sandboxes, communication protocols, or domain-specific evaluation systems. Instead, it gives developers a practical way to organize the work that happens before and around those systems.

This contribution is useful precisely because coding-AI development is increasingly hybrid. Human developers set goals, approve assumptions, and judge quality. Coding AIs inspect files, propose structures, implement changes, and run checks. Deterministic code validates schemas, renders review pages, and executes reproducible computations. Git records how all of these pieces evolve. `agentr` makes this hybrid arrangement more explicit by giving each part a place in the workflow.

Conclusion

`agentr` offers a practical answer to a specific problem: how to rely heavily on external coding AIs without losing structural control, transparency, provenance, or human authority. It does so by turning intermediate design into inspectable files and review artifacts. The result is a middle layer between manual development and opaque automation.

For the right users and tasks, this middle layer is valuable. It lets coding AIs perform substantial implementation work while keeping the task design explicit. It lets humans review workflows, memory, knowledge, assumptions, autonomy boundaries, and visual summaries before execution. It lets teams use Git to track how both code and design evolve. And it lets downstream runtimes remain flexible because the core objects are portable specifications rather than provider-specific model calls.

The central claim of `agentr` is therefore modest but important: as coding AIs become more capable, developers need better scaffolding around them. `agentr` provides that scaffolding for R-centered, repository-based, review-sensitive workflows.

References

- Amini, Soufiane, Yassine Benajiba, Cesare Bernardis, Paul Cayet, Hassan Chafi, Abderrahim Fathan, Louis Faucon, et al. 2025. “Open Agent Specification (Agent Spec): A Unified Representation for AI Agents.” <https://arxiv.org/abs/2510.04173v4>.
- Bandara, Eranga, Ross Gore, Peter Foytik, Sachin Shetty, Ravi Mukkamala, Abdul Rahman, Xueping Liang, et al. 2025. “A Practical Guide for Designing, Developing, and Deploying Production-Grade Agentic AI Workflows.” <https://arxiv.org/abs/2512.08769v1>.
- Barenji, Reza Vatankhah, and Sina Khoshgoftar. 2025. “Agentic AI for Autonomous Anomaly Management in Complex Systems.” <https://arxiv.org/abs/2507.15676v1>.
- Derouiche, Hana, Zaki Brahmi, and Haithem Mazeni. 2025. “Agentic AI Frameworks: Architectures, Protocols, and Design Challenges.” <https://arxiv.org/abs/2508.10146v1>.
- Du, Pengfei. 2026. “Memory for Autonomous LLM Agents: mechanisms, Evaluation, and Emerging Frontiers.” <https://arxiv.org/abs/2603.07670v1>.
- Dumas, Marlon, Fredrik Milani, and David Chapela-Campa. 2026. “Agentic Business Process Management Systems.” <https://arxiv.org/abs/2601.18833v1>.
- Feng, K. J. Kevin, David W. McDonald, and Amy X. Zhang. 2025. “Levels of Autonomy for AI Agents.” <https://arxiv.org/abs/2506.12469v2>.
- Ferrag, Mohamed Amine, Norbert Tihanyi, and Merouane Debbah. 2025. “From LLM Reasoning to Autonomous AI Agents: A Comprehensive Review.” <https://arxiv.org/abs/2504.19678v2>.
- Ghosh, Shaona, Barnaby Simkin, Kyriacos Shiarlis, Soumili Nandi, Dan Zhao, Matthew Fiedler, Julia Bazinska, et al. 2025. “A Safety and Security Framework for Real-World Agentic Systems.” <https://arxiv.org/abs/2511.21990v1>.
- Huang, Xu, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. “Understanding the Planning of LLM Agents: A Survey.” <https://arxiv.org/abs/2402.02716v1>.

- Kamalov, Firuz, David Santandreu Calonge, Linda Smail, Dilshod Azizov, Dimple R. Thadani, Theresa Kwong, and Amara Atif. 2025. "Evolution of AI in Education: Agentic Workflows." <https://arxiv.org/abs/2504.20082v2>.
- Kubam, Chandra Sekhar. 2025. "Agentic AI for Autonomous, Explainable, and Real-Time Credit Risk Decision-Making." <https://doi.org/10.5281/zenodo.18008872>.
- Lazer, Sahaya Jestus, Kshitiz Aryal, Maanak Gupta, and Elisa Bertino. 2026. "A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-Case Prototypes." <https://arxiv.org/abs/2601.05293v1>.
- Milosevic, Zoran, and Fethi Rabhi. 2026. "Architecting Agentic Communities Using Design Patterns." <https://arxiv.org/abs/2601.03624v2>.
- Naqvi, Saba, Mohammad Baqar, and Nawaz Ali Mohammad. 2026. "The Rise of Agentic Testing: Multi-Agent Systems for Robust Software Quality Assurance." <https://arxiv.org/abs/2601.02454v1>.
- Ning, Xuying, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, et al. 2026. "Code as Agent Harness." <https://arxiv.org/abs/2605.18747v1>.
- Uulu, Choro Ulan, Mikhail Kulyabin, Iris Fuhrmann, Jan Joosten, Nuno Miguel Martins Pacheco, Filippos Petridis, Rebecca Johnson, Jan Bosch, and Helena Holmström Olsson. 2026. "How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework." <https://arxiv.org/abs/2601.15153v1>.
- Wang, Zora Zhiruo, Yijia Shao, Omar Shaikh, Daniel Fried, Graham Neubig, and Diyi Yang. 2025. "How Do AI Agents Do Human Work? Comparing AI and Human Workflows Across Diverse Occupations." <https://arxiv.org/abs/2510.22780v2>.
- Wray, Robert E., James R. Kirk, and John E. Laird. 2025. "Applying Cognitive Design Patterns to General LLM Agents." <https://arxiv.org/abs/2505.07087v2>.
- Xi, Zhiheng, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, et al. 2023. "The Rise and Potential of Large Language Model Based Agents: A Survey." <https://arxiv.org/abs/2309.07864v3>.
- Yang, John, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. "SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering." <https://arxiv.org/abs/2405.15793v3>.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. "ReAct: Synergizing Reasoning and Acting in Language Models." <https://arxiv.org/abs/2210.03629v3>.
- Zhang, Gaoyang, Shanghong Zou, Yafang Wang, He Zhang, Ruohua Xu, and Feng Zhao. 2026. "ReusStd-Flow: A Standardized Reusability Framework for Dynamic Workflow Construction in Agentic AI." <https://arxiv.org/abs/2602.14922v1>.
- Zhang, Guangwei. 2025. "Knowledge Protocol Engineering: A New Paradigm for AI in Domain-Specific Knowledge Work." <https://arxiv.org/abs/2507.02760v1>.
- Zhu, Judy, Dhari Gandhi, Himanshu Joshi, Ahmad Rezaie Mianroodi, Sedef Akinli Kocak, and Dhanesh Ramachandran. 2026. "Interpreting Agentic Systems: Beyond Model Explanations to System-Level Accountability." <https://arxiv.org/abs/2601.17168v1>.
- Zou, Henry Peng, Wei-Chieh Huang, Yaozu Wu, Chunyu Miao, Dongyuan Li, Aiwei Liu, Yue Zhou, et al. 2025. "A Call for Collaborative Intelligence: Why Human-Agent Systems Should Precede AI Auton-

omy.” <https://arxiv.org/abs/2506.09420v1>.